

The layout of directory data blocks

במערכת הקבצים ext4 (בסביבת Linux Kernel 6.14), תיקייה (Directory) נתפסת מבחינה מבנית כקובץ לכל דבר ועניין. המשמעות היא שלתיקייה יש Inode משלה, וה-Inode הזה מצביע ל-Data Blocks בדיוק כמו קובץ טקסט או תוכנה.

ההבדל הקרדינלי אינו במבנה ה-Inode, אלא בתוכן שנשמר בתוך ה-Data Blocks של התיקייה: במקום תוכן גולמי של קובץ, הבלוקים של התיקייה מכילים רשימה של רשומות המקשרות בין שמות הקבצים לבין מספרי ה-Inodes שלהם. רשומות אלו נקראות Directory Entries (ובגרסת ext4 המודרנית: ext4_dir_entry_2).

להלן ניתוח מדויק, מקיף ומעמיק של ה-Layout של בלוק הנתונים של תיקייה בתוך מרחב ה-KB4 של ה-SSD.

מבנה כללי של בלוק נתונים של תיקייה (Directory Data Block Layout)

בלוק נתונים סטנדרטי של תיקייה בגודל 4096 בייט (KB4) אינו מחולק לתאים קבועים מראש, אלא בנוי כרצף צפוף של רשומות באורכים משתנים (Variable-length Records) הארוזות זו אחר זו בצורה לינארית.

מבנה הזיכרון בתוך בלוק ה-KB4 נראה כך:

```
[ Entry 1 (., Inode X) ] -> [ Entry 2 (.., Inode Y) ] -> [ Entry 3 (file.txt, Inode
```

המבנה הפנימי של רשומה בודדת (struct ext4_dir_entry_2)

גודלו של מבנה הנתונים הזה הוא משתנה, והוא מורכב מהשדות הבאים:

שם השדה בקרנל	גודל השדה (בבייטים)	תיאור תמציתי
inode	4 בייט (32 ביט)	מספר ה-Inode של הקובץ או תת-התיקייה המקושרים.
rec_len	2 בייט (16 ביט)	אורך הרשומה הכולל (Record Length) בבייטים.

שם השדה בקרנל	גודל השדה (בבייטים)	תיאור תמציתי
name_len	1 בייט (8 ביט)	אורך שם הקובץ בפועל (בבייטים).
file_type	1 בייט (8 ביט)	סוג האובייקט (קובץ רגיל, תיקייה, לינק וכו').
name	משתנה (1 עד 255 בייט)	מחרוזת השם של הקובץ, מיושרת לכפולות של 4 בייט.

הסבר מפורט וניתוח מעמיק של כל שדה ברשומה

א. השדה inode (4 בייט)

- משמעות הערך:** מספר שלם אי-שלילי המציין את מספר ה-Inode שבו שמור המטא-דאטה של הקובץ או התיקייה הנושאת את השם הזה.
- חשיבות לקרנל:** כאשר משתמש מבקש לגשת לקובץ בתוך תיקייה (למשל פתיחת /etc/passwd), הקרנל קורא את בלוק הנתונים של תיקיית השורש, סורק את שמות הקבצים עד שהוא מוצא את השם התואם, ושולף מתוך שדה ה-inode את המספר (לדוגמה: Inode 12345). בעזרת מספר זה הקרנל פונה ישירות לטבלת ה-Inodes כדי לטעון את מטא-דאטה של הקובץ.
- מקרה קצה (מחיקה לוגית):** אם שדה ה-inode מאופס (0), הדעיכה מעידה שהרשומה הזו ריקה או נמחקה, והקרנל מתעלם ממנה בזמן סריקת התיקייה.

ב. השדה rec_len (2 - Record Length בייט)

- משמעות הערך:** האורך הכולל בבייטים של הרשומה הנוכחית, המאפשר לקרנל לדעת היכן מתחילה הרשומה הבאה בבלוק.
- למה שדה זה קריטי?** היות ששמות קבצים הם באורכים משתנים (שם של תווים ספורים לעומת שם ארוך), הרשומות אינן בגודל קבוע. הקרנל אינו יכול "לקפוץ" בצעדים קבועים. כדי לעבור מרשומה א' לרשומה ב', הקרנל לוקח את כתובת הזיכרון של תחילת הרשומה הנוכחית ומוסיף אליה את הערך של rec_len.
- כלל היישור (Alignment):** הערך של rec_len חייב להיות תמיד מספר זוגי ומתואם (Aligned) לכפולות של 4 בייט.
- הרשומה האחרונה בבלוק:** הרשומה האחרונה בבלוק ה-KB4 מחזיקה בערך rec_len שאינו מתאר רק את גודלה האמיתי, אלא **משתרע עד לסוף בלוק ה-KB4 המלא**, ובכך ממלא את כל יתרת הנפח הפנוי שנותר בבלוק.

ג. השדה name_len (1 בייט)

- משמעות הערך:** אורך המחרוזת של שם הקובץ בפועל בבייטים (למשל, עבור השם main.c, הערך יהיה 6).
- חשיבות לקרנל:** מאפשר לקרנל לדעת כמה תווים בדיוק לקרוא מתוך מחרוזת השם (name) מבלי להסתמך בהכרח על תו סיום מחרוזת (0), ובכך לייעל את תהליך ההשוואה כאשר מחפשים קובץ בתיקייה.

ד. השדה file_type (1 בייט)

- משמעות הערך:** מציין את סוג האובייקט (למשל: קובץ רגיל DT_REG, תיקייה DT_DIR, קישור סימבולי DT_LNK).
- חשיבות ארכיטקטונית ב-ext4:** במערכות קבצים ישנות יותר (כמו ext2), הקרנל נאלץ לקרוא את בלוק התיקייה, למצוא את מספר ה-Inode, ולגשת פיזית אל טבלת ה-Inodes.

בדיסק כדי לבדוק את ה-`i_mode` ולגלות האם מדובר בקובץ או בתיקייה. ב-`ext4`, שמירת סוג האובייקט ישירות בתוך ה-`Directory Entry` חוסכת לחלוטין את הגישה הנוספת לטבלת ה-`Inodes` ומאיצה דרסטית פקודות מערכת כמו `ls -l` או סריקות עצי ספריות.

E. השדה `name` (משתנה, 1 עד 255 בייט)

- **משמעות הערך:** מחרוזת הטקסט המכילה את שם הקובץ או התיקייה.
- **מבנה הזיכרון ויישור (Padding):** מכיוון שכל רשומה בשרשרת חייבת להתחיל בכתובת המסתיימת בכפולות של 4 בייט, לאחר סיום מחרוזת השם מוסיפים בקרנל בייטים ריקים (Padding) עד להשלמת היישור הנדרש עבור ה-`rec_len` של הרשומה הבאה.

4. ניהול מחיקה בתוך בלוק התיקייה (איך עובד `rm`?)

מנגנון המחיקה של קובץ מתוך תיקייה מדגים בצורה מושלמת את חשיבות שדה ה-`rec_len`:

1. כאשר משתמש מוחק קובץ מתוך תיקייה (באמצעות פקודת `unlink`), הקרנל **אינו** מזיז את כל שאר הרשומות בבלוק ולא מאפס את כל הבייטים מחדש (פעולה שהייתה כבדה ואיטית).
2. במקום זאת, הקרנל מאפס את שדה ה-`inode` של אותה רשומה ל-0 (ומסמן אותה כלא תקפה).
3. לאחר מכן, הקרנל מנווט אל הרשומה **הקודמת** לה בשרשרת, ומעדכן את ה-`rec_len` שלה כך שהוא "יבלע" לתוכו גם את השטח של הרשומה שנמחקה.
4. באופן זה, השטח של הקובץ שנמחק הופך לחלק מה-`Padding` של הרשומה הקודמת, ומשמש כ"מקום פנוי" (`Free Space`) שאליו יוכל להיכנס בעתיד קובץ חדש שיוצר באותה תיקייה אם גודלו יתאים למרווח שנוצר.

5. הצירופים המיוחדים: 1. -..

כל בלוק ראשון של תיקייה חדשה שנוצרת במערכת הקבצים חייב להכיל תמיד שתי רשומות קבועות ומיוחדות בתחילתו:

1. **רשומת (Current Directory):** מצביעה באמצעות שדה ה-`inode` שלה ישירות על מספר ה-`Inode` של ה**תיקייה עצמה**.
2. **רשומת .. (Parent Directory):** מצביעה באמצעות שדה ה-`inode` שלה על מספר ה-`Inode` של ה**תיקייה האם** (התיקייה המכילה אותה).

רשומות אלו נבנות על ידי ה-FHS/הקרנל ברגע יצירת התיקייה (`sys_mkdir`), והן מאפשרות לתהליכי מערכת ההפעלה לנוע במרחב הספריות קדימה ואחורה בצורה דטרמיניסטית וללא תלות בחישוב חיצוני.

למה בפועל צריך את השדה `rec_len`?

הרי ניתן לחשב היכן מתחילה הרשומה הבאה על סמך העובדה שידוע לקרנל הגודל בבתים של כל השדות.
כולל האורך בבתים של שם הקובץ. שהרי הוא שמור בשדה `name`.

השאלה הזו חושפת נקודה ארכיטקטונית מרתקת ויסודית בתכנון מערכות קבצים. ההגיון לכאורה מושלם: אם ידועים לנו הגדלים הקבועים של השדות (`rec_len`, `inode`), ואנחנו יודעים בדיוק את אורך מחרוזת השם (`name_len`), לכאורה אפשר לחשב מתמטית את גודל הרשומה הנוכחית ולדעת היכן מתחילה הבאה.

אולם, ישנן שתי סיבות קריטיות – אחת מבוססת על ביצועי חומרה ויישור זיכרון (**Alignment**) והשנייה מבוססת על ניהול מקום פנוי במחיקות ועדכוני (**Deletion & Fragmentation**) – שבגללן השדה `rec_len` הוא הכרחי לחלוטין.

1. בעיית היישור (Alignment) והריפוד (Padding)
בארכיטקטורות מעבדים מודרניות (כמו `x86_64` שעליה רץ `Linux Kernel 6.14`), גישה לזיכרון או קריאה של משתנים בכתובות שאינן מיושרות (`Unassigned/Misaligned Memory Access`) גורמת לעונשי ביצועים חמורים או אפילו לשגיאות חומרה (`Bus Fault`).

- כל רשומה במבנה התיקיה חייבת להתחיל בכתובת זיכרון שהיא כפולה של 4 בייט.
- משמעות הדבר היא שאם אורך שם הקובץ הוא אי-זוגי (למשל, 3 בייט לשם `abc`), אי אפשר פשוט להצמיד את הרשומה הבאה מיד אחריו, מכיוון שזה היה שובר את כלל היישור של ה-4 בייט.
- לכן, מוסיפים לסוף השם בייטים ריקים סמויים (`Padding`).
- התפקיד של `rec_len`:** הגודל התיאורטי של השדות ועוד אורך השם אינו בהכרח הגודל האמיתי שתופסת הרשומה בפועל בזיכרון, כי הוא כולל בתוכו גם את בייטי הריפוד (`Padding`) שנוספו לצורך יישור. `rec_len` הוא המספר האמיתי והמדויק שמודיע לקרנל: "כדי להגיע לתחילת הרשומה הבאה במרחב הזיכרון המיושר, עליך לקפוץ בדיוק `X` בייטים קדימה", בלי שהקרנל יצטרך לחשב בכל פעם מחדש את חוקיות הריפוד.

2. ניהול מקום פנוי ומחיקות (הסיבה המרכזית)
כפי שראינו בשאלת המחיקה (`unlink / rm`), כאשר קובץ נמחק מתוך תיקיה, הקרנל אינו מזיז את כל שאר הרשומות בבלוק ולא דוחס את הזיכרון (פעולה שהייתה כבדה, איטית ומחייבת עדכון של אינספור פוינטרים).

- במקום זאת, מבוצעת "מחיקה לוגית": שדה ה-`inode` של הרשומה שנמחקה מאופס ל-0, והרשומה הזו הופכת ל"שטח מת" (`Dead Space / Hole`) בתוך בלוק ה-4KB.
- כדי שהשטח הזה לא יתבזבז, הקרנל מעדכן את שדה ה-`rec_len` של הרשומה הקודמת לה בשרשרת, כך שהוא יגדל ויבלע לתוכו את השטח של הרשומה שנמחקה.
- המשמעות הקריטית:** במצב כזה, הגודל הפיזי של הרשומה בפועל (`rec_len`) כבר אינו תואם כלל לגודל התיאורטי של השדות ושם הקובץ שלה! `rec_len` הפך לגדול במתכוון כדי "לדלג" מעל השטח הפנוי ולהצביע ישירות על הרשומה החוקית הבאה שקיימת אחריו בבלוק.

ללא `rec_len`, הקרנל לא היה מסוגל לנוע ברשת הרשומות המשתנות האלו, לא היה יודע להתעלם משטחים מתים שנמחקו, ולא היה יכול לנצל מחדש שטחים פנויים בתוך בלוק התיקיה מבלי לבצע פעולות דחיסה (`Defragmentation`) כבדות בכל מחיקת קובץ.

